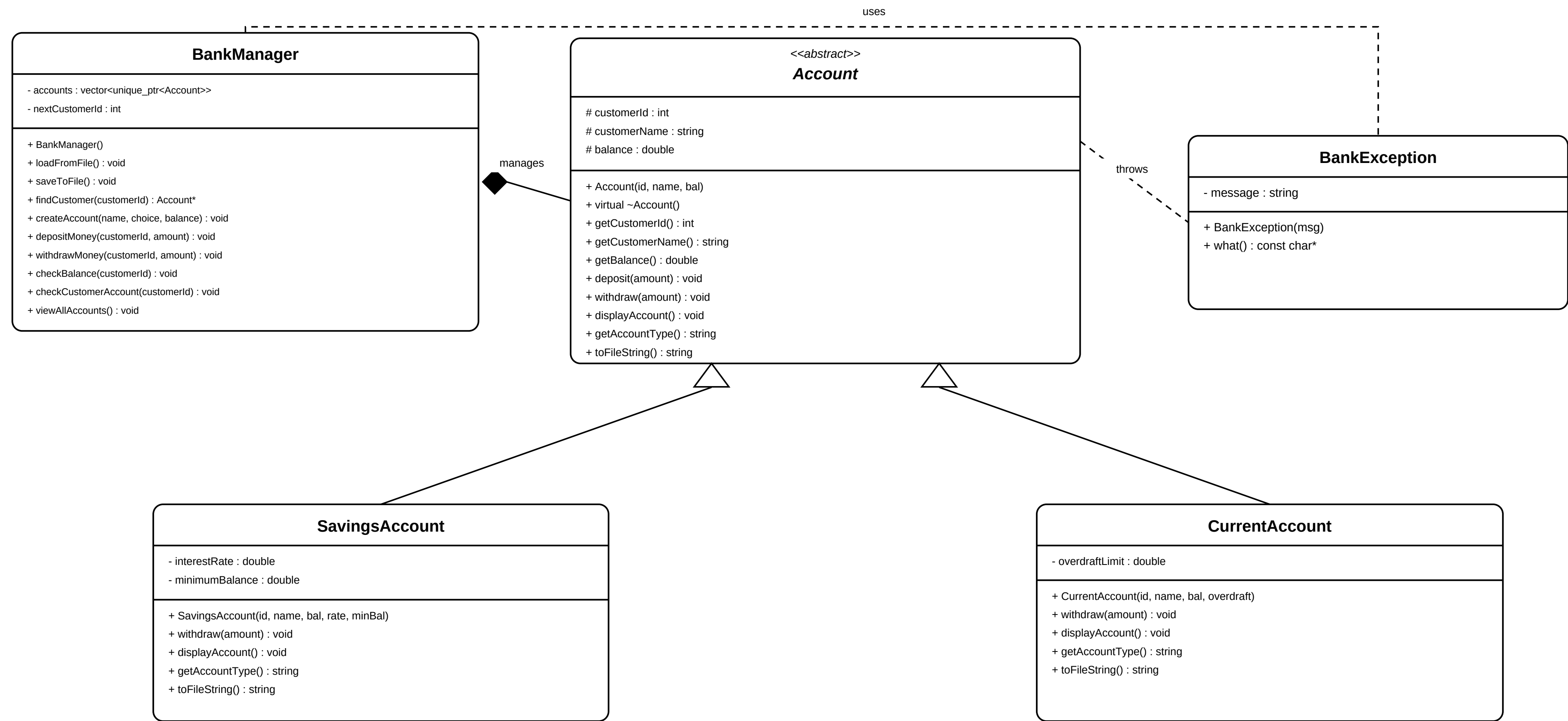


Bank Account Manager - UML Class Diagram



Legend

+ public - private # protected

Hollow triangle arrow: inheritance

Filled diamond/line: BankManager contains/manages Account objects

Dashed line: dependency, uses, or throws exception

Main Relationships

- SavingsAccount is an Account
- CurrentAccount is an Account
- BankManager manages Account objects using vector<unique_ptr<Account>>
- BankException is used for custom error handling

Bank Account Manager - UML Class Diagram

OOP II Final Project | CLI-based banking system

Diagram Explanation

Account:	Abstract base class. Stores common account data and declares virtual functions.
SavingsAccount:	Derived class of Account. Adds interest rate and minimum balance. Has its own withdraw rule.
CurrentAccount:	Derived class of Account. Adds overdraft limit. Has its own withdraw rule.
BankManager:	Manager class. Stores a list of Account pointers and controls create, deposit, withdraw, balance check, file save/load.
BankException:	Custom exception class used for meaningful error handling.

OOP Concepts Shown

Encapsulation:	customerId, customerName, and balance are protected in Account and accessed through methods.
Inheritance:	SavingsAccount and CurrentAccount inherit from Account.
Abstraction:	Account is abstract because it has pure virtual functions.
Runtime Polymorphism:	BankManager stores vector<unique_ptr<Account>> and calls virtual functions like withdraw() and displayAccount().
Exception Handling:	BankException is used for invalid input, missing customer, and withdrawal rule errors.
File I/O:	BankManager saves and reloads account data from data/accounts.txt.

Final Project Notes

- The project is entirely CLI-based.
- Each customer has one account type: Savings or Current.
- Account data is saved and loaded using file input/output.
- The diagram shows major classes, attributes, methods, and relationships.